Supervyse OPF Intel Firmware Update Porting Guide

Revision 1.3

12 2024

Copyright (c) 2024, All Rights Reserved. Insyde Software.

No part of this publication may be reproduced, transmitted, transcribed, stored in a retrieval system, or translated into any language or computer language, in any form, or by any means, electronic, mechanical, magnetic, optical, chemical, manual or otherwise, without the prior written permission of Insyde Software Corp.

Disclaimer

Insyde Software provides this document and the programs "as is" without warranty of any kind, either expressed or implied, including, but not limited to, the implied warranties of merchantability or fitness for a particular purpose.

This document could contain technical inaccuracies or typographical errors. Changes are periodically made to the information herein; these changes will be incorporated in future revisions of this document. Insyde Software is under no obligation to notify any person of the changes.

The following trademarks are used in this document:

Insyde and InsydeH2O are registered trademarks or trademarks of Insyde Software. All other trademarks or trade names are property of their respective holders.

Contents

1. Introduction 5
2. Image Packing - Windows 6
   - 2.1 Tarball Format 6
     - 2.1.1 Tool Information 6
     - 2.1.2 How to Install 7
     - 2.1.3 How to Generation Image Tarball 7
       - 2.1.3.1. BMC 7
       - 2.1.3.2. BIOS 7
       - 2.1.3.3. CPLD 8
   - 2.2 HPM Format 8
     - 2.2.1 Tool Information 8
     - 2.2.2 How to Install **Error! Bookmark not defined.**
     - 2.2.3 How to Generation HPM Image 8
       - 2.2.3.1. BMC 9
       - 2.2.3.2. BIOS 9
       - 2.2.3.3. CPLD 9
3. Image Packing - Linux 10
   - 3.1 Tarball Format 10
     - 3.1.1 Tool Information 10
     - 3.1.2 How to Generation Image Tarball 11
       - 3.1.2.1. BMC 11
       - 3.1.2.2. BIOS 11
       - 3.1.2.3. CPLD 11
   - 3.2 HPM Format 12
     - 3.2.1 Tool Information 12
     - 3.2.2 How to Generation HPM Image 12
       - 3.2.2.1. BMC 12
       - 3.2.2.2. BIOS 13
       - 3.2.2.3. CPLD 13
4. Signing BMC Firmware 14
   - 4.1 With Default Private Key 14
   - 4.2 With Specific Private Key 14

Revision History

Revision Number	Date	Author	Reviewer	Summary
1.0	2022-10-01	Sam Wu	Conrad Yao	Initial version
1.1	2024-08-28	Sam Wu	Conrad Yao	Add Tool Information
1.2	2024-12-09	Sam Wu	Conrad Yao	Revise some contents
1.3	2025-01-22	Ben Chang	Conrad Yao	Add a description about CPLD not being supported.

1. Introduction

This page introduces the following topics that OPF uses and guides you how to use firmware update relative functions.

- Image Packing
  - Tarball format
  - HPM format
- Image Signing

2. Image Packing - Windows

Currently, there are two types of firmware images supported for updates within the OPF: tarball format, which can be updated through BMCWeb and Redfish interfaces, and HPM format, which can be updated using the IPMI protocol stack.

- Tarball for BMCWeb and Redfish interfaces
- HPM for IPMI protocol stack via spv_ipmi tool

2.1 Tarball Format

Firmware images can come in various types, such as binary, raw data, HPM, PLDM type 5, and so on. To standardize the firmware update process and ensure consistency, all firmware images within the OPF must be packaged as tarballs. For instance, bios.tar.gz or fpga.tar.gz. This unified approach allows all firmware updates to be initiated through BMCWeb or Redfish interfaces, and the BMC can then determine the specific type of firmware image and perform the corresponding update actions.

2.1.1 How to Install

The Firmware Image Tarball Generator is a shell script tool designed to generate firmware image tarballs.
The tool is distributed as an installer package named **gen_image_tar_installer.exe.**

- Double click gen_image_tar_installer.exe to install the tool.
- The default installation path is
  C:\Users\{UserName}\AppData\Local\Insyde\gen-image-tar
- Ensure that the installation path is added to the PATH environment variable

2.1.2 Tool Information

```
usage: gen-image-tar.exe [OPTION] <Image FILE>...
```

```
Generates an image tarball from given file as input.
Creates a MANIFEST for image verification and recreation
Packages the image and MANIFEST together in a tarball

usage: gen-image-tar [OPTION] <Image FILE>...

Options:
  -o, --out <file>       Specify destination file. Defaults to
                       `pwd`/obmc-image.tar.gz if unspecified.
  -s, --sign <path>      Sign the image. The optional path argument specifies
                       the private key file. Defaults to the bash variable
                       PRIVATE_KEY_PATH if available, or else uses the
                       open-source private key in this script.
  -m, --machine <name>   Specify the target machine name of this
                       image.
  -v, --version <name>   Specify the version of image file
  -t, --type <bios/cpld/erot/fpga/vr/retimer/scm-fpga/hpm-fpga> Specify what type of file.
  -u, --script <path>    Optionally specify the update script
  -h, --help             Display this help text and exit.
```

Parameters can be either optional or mandatory. Mandatory parameters must be specified; otherwise, an error will occur when executing the tool.

- The -o parameter is optional
- The -s parameter is optional
- The -u parameter is deprecated
- The -m parameter is **mandatory**
- The -v parameter is **mandatory**
- The -t parameter is **mandatory**

2.1.3 HHow to Generation Image Tarball

- Prerequisites
  - Tool: gen-image-tar.exe
  - OpenBMC.priv key file or any other private key file
  - Firmware image file

The following instructions demonstrate how to use the tool to generate a firmware image tarball for different types of firmware. If you decompress the tarball firmware image, there are some files within it:

- MANIFEST
  - used to describe software package metadata, including version, type, and encryption algorithm
- image-*
  - firmware images
- publickey
  - used to validate the digital signature associated with firmware images
- image*.sig
  - used to contain the digital signature which is used to authenticate the firmware image

```
image-kernel
image-kernel.sig
image-rofs
image-rofs.sig
image-rwfs
image-rwfs.sig
image-sdk-info
image-sdk-info.sig
image-u-boot
image-u-boot.sig
MANIFEST
MANIFEST.sig
publickey
publickey.sig
```

2.1.3.1. BMC

- BMC firmware image tarball is produced by compiling OPF SDK. As a result, there's no need for extra steps to create the BMC firmware image tarball.
  - The location of BMC firmware image tarball is under

```
SDK/build/<platform_name>/tmp/deploy/images/<platform_name>/image-update.tar
```

2.1.3.2. BIOS

- Intel's BIOS firmware images need to be stored in a HPM format first, please refer to the subsequent chapters for guidance on creating an image in HPM format.
- Please pay special attention to the "-t" parameter, which must be specified as "bios".
- Steps To Generate Image Tarball File
  - Ensure the required tools and private key are prepared

- Verify the input file is in the working directory
- Open Command Prompt (cmd / PowerShell) and navigate to working directory
- Run the following command to generate the Tarball File

```
gen-image-tar.exe -m sp5-insyde -v v1.34 -t bios syde-bios.FD -o obmc-image-
bios.tar.gz
```

2.1.3.3. CPLD

- **Intel BHS CRB and EGS CRB does not support CPLD update**.

2.2HPM Format

In addition to BMCWeb and Redfish updates (.tar.gz tarball format) within the OPF, updates can also be performed using the spv_ipmi tool, which leverages the IPMI protocol (hpm format). This section will introduce how to package HPM format firmware images.

2.2.1 How to Install

The HPM Firmware Generator is a Python-based tool designed to generate HPM firmware images.
The tool is distributed as an installer package named **GenHpmFw_installer.exe.**

- Double click GenHpmFw_installer.exe to install the tool.
- The default installation path is C:\Users\{UserName}\AppData\Local\Insyde\hpmtool-windows
- Ensure that the installation path is added to the PATH environment variable

2.2.2 Tool Information

```
GenHpmFw.exe
Need a config file
```

For Intel platform, there are two config files that can be used for generating HPM images:

- This is for BMC to use.

```
hpmtool-windows/Configs/HPM_BMC.conf
```

- This is for BIOS to use.

```
hpmtool-windows/Configs/HPM_BIOS.conf
```

2.2.3 How to Generation HPM Image

- Prerequisites
  - Tool: GenHpmFw.exe
  - Firmware image file
  - The configuration file

Please note the following configuration requirements:

- The "Firmware-Name" field should be adjusted to point to the input file. The purpose of the "HPM-Image-Name" field is to specify the output file, which will be in the HPM format.
- The Firmware-Name in the configuration file must point to a tarball. Please follow the guidelines in the previous chapter to create the necessary tarball firmware image.

For instance, this configuration file example (HPM_BMC.conf) is for BMC HPM firmware image to use.

```
### Upgrade Component ###
Upgrade-Component-Name=BMC

### Output HPM Image Name ###
HPM-Image-Name=./bmc.hpm

### Input Source Firmware ###
# BMC Firmware Name #
Firmware-Name=./obmc-image_BMC.tar
```

2.2.3.1. BMC

- Steps To Generate Image HPM File
  - Ensure the required tools and configuration file are prepared
  - Verify the input firmware image file is in the working directory
  - Open Command Prompt (cmd / PowerShell) and navigate to working directory
  - HPM firmware image for BMC will be produced by executing the following command.

```
GenHpmFw.exe -conf ./HPM_BMC.conf
```

2.2.3.2. BIOS

- Steps To Generate Image HPM File
  - Ensure the required tools and configuration file are prepared
  - Verify the input firmware image file is in the working directory
  - Open Command Prompt (cmd / PowerShell) and navigate to working directory
  - HPM firmware image for BIOS will be produced by executing the following command.

```
GenHpmFw.exe -conf ./HPM_BIOS.conf
```

2.2.3.3. CPLD

- **Intel BHS CRB and EGS CRB does not support CPLD update.**

3. Image Packing - Linux

Currently, there are two types of firmware images supported for updates within the OPF: tarball format, which can be updated through BMCWeb and Redfish interfaces, and HPM format, which can be updated using the IPMI protocol stack.

- Tarball for BMCWeb and Redfish interfaces
- HPM for IPMI protocol stack via spv_ipmi tool

3.1 Tarball Format

Firmware images can come in various types, such as binary, raw data, HPM, PLDM type 5, and so on. To standardize the firmware update process and ensure consistency, all firmware images within the OPF must be packaged as tarballs. For instance, bios.tar.gz or fpga.tar.gz. This unified approach allows all firmware updates to be initiated through BMCWeb or Redfish interfaces, and the BMC can then determine the specific type of firmware image and perform the corresponding update actions.

3.1.1 Tool Information

The Firmware Image Tarball Generator is a shell script tool designed to generate firmware image tarballs.
The tool is located at

```
SDK/meta-insyde/tools/gen-image-tar
```

```
usage: gen-image-tar [OPTION] <Image FILE>...
```

```
Generates an image tarball from given file as input.
Creates a MANIFEST for image verification and recreation
Packages the image and MANIFEST together in a tarball

usage: gen-image-tar [OPTION] <Image FILE>...

Options:
  -o, --out <file>       Specify destination file. Defaults to
                         `pwd`/obmc-image.tar.gz if unspecified.
  -s, --sign <path>      Sign the image. The optional path argument specifies
                         the private key file. Defaults to the bash variable
                         PRIVATE_KEY_PATH if available, or else uses the
                         open-source private key in this script.
  -m, --machine <name>   Specify the target machine name of this
                         image.
  -v, --version <name>   Specify the version of image file
  -t, --type <bios/cpld/erot/fpga/vr/retimer/scm-fpga/hpm-fpga> Specify what type of file.
  -u, --script <path>    Optionally specify the update script
  -h, --help             Display this help text and exit.
```

Parameters can be either optional or mandatory. Mandatory parameters must be specified; otherwise, an error will occur when executing the tool.

- The -o parameter is optional
- The -s parameter is optional
- The -u parameter is deprecated
- The -m parameter is **mandatory**
- The -v parameter is **mandatory**

- The -t parameter is **mandatory**

3.1.2 How to Generation Image Tarball

- Prerequisites
  - Tool: gen-image-tar
  - OpenBMC.priv key file or any other private key file
  - Firmware image file

The following instructions demonstrate how to use the tool to generate a firmware image tarball for different types of firmware. If you decompress the tarball firmware image, there are some files within it:

- MANIFEST
  - used to describe software package metadata, including version, type, and encryption algorithm
- image-*
  - firmware images
- publickey
  - used to validate the digital signature associated with firmware images
- image*.sig
  - used to contain the digital signature which is used to authenticate the firmware image

```
image-kernel
image-kernel.sig
image-rofs
image-rofs.sig
image-rwfs
image-rwfs.sig
image-sdk-info
image-sdk-info.sig
image-u-boot
image-u-boot.sig
MANIFEST
MANIFEST.sig
publickey
publickey.sig
```

3.1.2.1. BMC

- BMC firmware image tarball is produced by compiling OPF SDK. As a result, there's no need for extra steps to create the BMC firmware image tarball.
  - The location of BMC firmware image tarball is under

```
SDK/build/<platform_name>/tmp/deploy/images/<platform_name>/image-update.tar
```

3.1.2.2. BIOS

- Intel's BIOS firmware images need to be stored in a HPM format first, please refer to the subsequent chapters for guidance on creating an image in HPM format.
- Please pay special attention to the "-t" parameter, which must be specified as "bios".

```
cd SDK/meta-insyde/tools/
./gen-image-tar -m intel-bhs-insyde -v 0061 -t bios bhs-0061.bin -o obmc-image-
bios.tar.gz
```

3.1.2.3. CPLD

- **Intel BHS CRB and EGS CRB does not support CPLD update**.

3.2HPM Format

In addition to BMCWeb and Redfish updates (.tar.gz tarball format) within the OPF, updates can also be performed using the spv_ipmi tool, which leverages the IPMI protocol (hpm format). This section will introduce how to package HPM format firmware images.

3.2.1 Tool Information

The HPM Firmware Generator is a Python-based tool designed to generate HPM firmware images.
The tool is located at

```
SDK/meta-insyde/tools/HPM/GenHpmFw.py
```

./GenHpmFw.py
Need a config file

For Intel platform, there are two config files that can be used for generating HPM images:

- This is for BMC to use.

SDK/meta-insyde/tools/HPM/Configs/HPM_BMC.conf

- This is for BIOS to use.

SDK/meta-insyde/tools/HPM/Configs/HPM_BIOS.conf

3.2.2 How to Generation HPM Image

- Prerequisites
  - Tool: GenHpmFw.py
  - Firmware image file
  - The configuration file

Please note the following configuration requirements:

- The "Firmware-Name" field should be adjusted to point to the input file. The purpose of the "HPM-Image-Name" field is to specify the output file, which will be in the HPM format.
- The Firmware-Name in the configuration file must point to a tarball. Please follow the guidelines in the previous chapter to create the necessary tarball firmware image.

```
For instance, this configuration file example (HPM_BMC.conf) is for BMC HPM firmware image to use.

### Upgrade Component ###
Upgrade-Component-Name=BMC

### Output HPM Image Name ###
HPM-Image-Name=./bmc.hpm

### Input Source Firmware ###


# BMC Firmware Name #
Firmware-Name=./obmc-image_BMC.tar
```

3.2.2.1. BMC

- HPM firmware image for BMC will be produced by executing the following command.

./GenHpmFw.py -conf ./HPM_BMC.conf

3.2.2.2. BIOS

- HPM firmware image for BIOS will be produced by executing the following command.

```
./GenHpmFw.py -conf ./Configs/HPM_BIOS.conf
```

3.2.2.3. CPLD

- **Intel BHS CRB and EGS CRB does not support CPLD update.**

4. Signing BMC Firmware

In order to secure the BMC, we need to sign all the images and include a public key in the package with which to verify future update images. OPF follows OpenBMC to add a framework to sign the image files with an open private key and generate a corresponding public key added to the image. This isn't secure by itself (since the private key is available), but additional changes can easily provide their own private key, creating a secure BMC.
The following are the points to be noted:

- RSA-SHA256 encryption is used.
- The private key will generate a public key, which will be placed in:
  - BMC rootfs /etc/activationdata/OpenBMC/.
  - The tarball firmware image.
- The public key will be used to decrypt the .sig file in the tarball firmware image.

4.1 With Default Private Key

- OPF uses the following default private key to sign BMC firmware.

```
<SDK>/meta-phosphor/recipes-phosphor/flash/files/OpenBMC.priv
```

- The publickey is used to verify whether the image is valid or not when firmware update is proceeding.
- The publickey corresponding to private key is in the following location.

```
<SDK>/build/tmp/work/$MACHINE-openbmc-linux-
gnueabi/$RECIPE/$VERSION/rootfs/etc/activationdata/key/publickey

EX:
build/tmp/work/starship_insyde-openbmc-linux-gnueabi/obmc-phosphor-image/1.0-
r0/rootfs/etc/activationdata/OpenBMC/publickey
```

- If you decompress the BMC firmware image, there are some .sig files which are used to be verified with publickey

```
 ...
image-kernel.sig
image-rofs.sig
image-rwfs.sig
image-sdk-info.sig
image-u-boot.sig
MANIFEST.sig
publickey.sig
 ...
```

4.2 With Specific Private Key

- If you want to use another private key, not the default one. You can use the following method to produce your own private key.
  - Generate RSA1024 private key named "OpenBMC.priv"

```
# openssl genrsa -out OpenBMC.priv 1024
```

  - Generate RSA2048 private key named "OpenBMC.priv"

```
# openssl genrsa -out OpenBMC.priv 2048
```

- Generate RSA4096 private key named "OpenBMC.priv"

```
# openssl genrsa -out OpenBMC.priv 4096
```

- Export some environment variables
- Build firmware image with environment variables

```
# export BB_ENV_EXTRAWHITE="$BB_ENV_EXTRAWHITE SIGNING_KEY"
# SIGNING_KEY=/path/to/secure/OpenBMC.priv bitbake obmc-phosphor-image
```